TinyTrack Performance Testing Report using k6

Overview:

Performance testing was conducted on the TinyTrack URL Shortener application to evaluate its scalability, reliability, and response time under different load conditions. The objective of this testing was to determine how many concurrent users the system can handle, identify bottlenecks, and analyze system behavior under stress.

Tools Used:

- k6 (Load Testing Tool)
- Node.js (Express Backend - Render)
- PostgreSQL (Neon Database)
- React Frontend (Vercel)

Test Setup

All performance tests were written using k6 and organized inside a dedicated folder (k6-tests/).

The tests were executed against the live production backend:

Base URL:
https://tinytrack-ydvz.onrender.com

Redirect Load Test Results

50 Users Load Test:

The redirect load test was executed with 50 concurrent users for 30 seconds.

The system successfully handled **474 requests** with a throughput of approximately **14 requests per second**.

All requests returned a valid **HTTP 301 redirect response**, resulting in a **100% success rate** and **0% error rate**, confirming correct functionality of the redirection mechanism.

```
TOTAL RESULTS

checks_total.......: 474     14.18239/s
checks_succeeded...: 100.00% 474 out of 474
checks_failed......: 0.00%   0 out of 474

✓ status is 301

HTTP
http_req_duration..............: avg=2.31s min=1.48s med=2.33s max=3.35s p(90)=2.42s p(95)=2.45s
  { expected_response:true }...: avg=2.31s min=1.48s med=2.33s max=3.35s p(90)=2.42s p(95)=2.45s
http_req_failed................: 0.00%  0 out of 474
http_reqs......................: 474    14.18239/s

EXECUTION
iteration_duration.............: avg=3.32s min=2.48s med=3.34s max=4.35s p(90)=3.43s p(95)=3.46s
iterations.....................: 474    14.18239/s
vus............................: 16     min=16       max=50
vus_max........................: 50     min=50       max=50

NETWORK
data_received..................: 527 kB 16 kB/s
data_sent......................: 106 kB 3.2 kB/s
```

The system achieved 100% success rate with no failures**, but response time was around 2.3 seconds.**

100 Users Load Test:

The system was tested with 100 concurrent users for 30 seconds. Again all requests were successfully processed with a **100% success rate** and **0% failure rate**, confirming system stability under increased load.

```
TOTAL RESULTS

  checks_total.......: 551     14.457093/s
  checks_succeeded...: 100.00% 551 out of 551
  checks_failed......: 0.00%   0 out of 551

  ✓ status is 301

  HTTP
  http_req_duration..............: avg=5.09s min=1.56s med=5.64s max=7.57s p(90)=6.37s p(95)=7.37s
    { expected_response:true }...: avg=5.09s min=1.56s med=5.64s max=7.57s p(90)=6.37s p(95)=7.37s
  http_req_failed................: 0.00%  0 out of 551
  http_reqs......................: 551    14.457093/s

  EXECUTION
  iteration_duration.............: avg=6.1s  min=2.62s med=6.65s max=8.58s p(90)=7.37s p(95)=8.38s
  iterations.....................: 551    14.457093/s
  vus............................: 1      min=1        max=100
  vus_max........................: 100    min=100      max=100

  NETWORK
  data_received..................: 815 kB 21 kB/s
  data_sent......................: 196 kB 5.2 kB/s


running (0m38.1s), 000/100 VUs, 551 complete and 0 interrupted iterations
default ✓ [======================================] 100 VUs  30s
PS C:\Users\jeffr\OneDrive\Desktop\url_shortner\k6-tests>
```

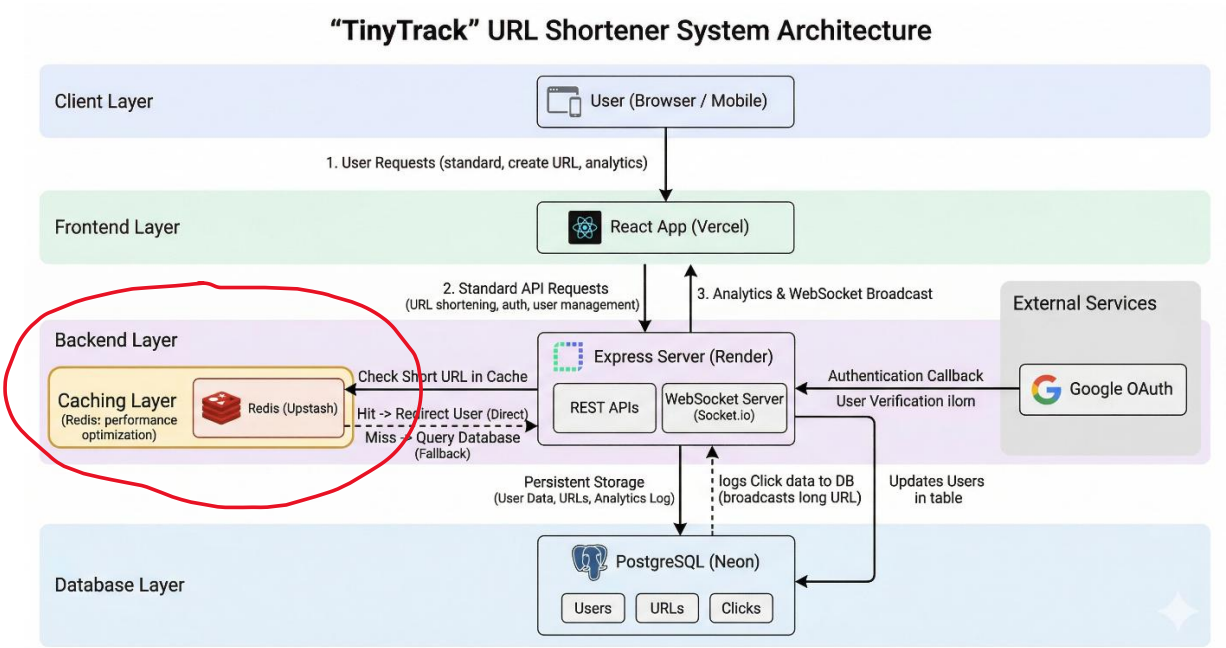
So my understanding is :

- Each redirect request triggers a database query.
- Lack of caching results in repeated database hits.
- Backend is running on a single instance (Render).
- Database (Neon) latency increases under load.

The system maintains stability under higher load but does not scale efficiently, as response time increases significantly without improvement in throughput. This confirmed me to add **caching layer.** So I change the architecture.

Performance Optimization of TinyTrack Using Redis Caching

In the previous architecture, every redirect required a database query, which **created a bottleneck under load**. Redis was introduced to reduce repeated database access for frequently requested short URLs and improve response speed. By caching these lookups, the system can serve requests faster and reduce backend pressure. This makes the application more scalable and efficient for read-heavy traffic patterns.



Results:

```
TOTAL RESULTS

    checks_total.......: 1817    57.336781/s
    checks_succeeded...: 100.00% 1817 out of 1817
    checks_failed......: 0.00%   0 out of 1817

    ✓ status is 301

    HTTP
    http_req_duration..............: avg=680.37ms min=132.54ms med=496.74ms max=4.86s p(9
0)=1.19s p(95)=1.69s
      { expected_response:true }...: avg=680.37ms min=132.54ms med=496.74ms max=4.86s p(9
0)=1.19s p(95)=1.69s
    http_req_failed................: 0.00%  0 out of 1817
    http_reqs......................: 1817   57.336781/s

    EXECUTION
    iteration_duration.............: avg=1.68s    min=1.13s    med=1.49s    max=5.93s p(9
0)=2.19s p(95)=2.72s
    iterations.....................: 1817   57.336781/s
    vus............................: 17     min=17        max=100
    vus_max........................: 100    min=100       max=100

    NETWORK
    data_received..................: 1.6 MB 51 kB/s
    data_sent......................: 246 kB 7.8 kB/s



running (0m31.7s), 000/100 VUs, 1817 complete and 0 interrupted iterations
default ✓ [======================================] 100 VUs  30s
PS C:\Users\jeffr\OneDrive\Desktop\url shortner\k6 tests>
```

Metric	Before Redis	After Redis
Avg Response Time	~5 seconds	**680 ms**
Throughput	~14 req/sec	**~57 req/sec**
Error Rate	0%	0%

The introduction of Redis caching significantly improved system performance by reducing dependency on the database for repeated requests.

The results clearly show:

- A **7x reduction in response time.**
- A **4x increase in throughput**.
- Consistent system stability under concurrent load.